Kick Start 2021 - Round B

Analysis: Consecutive Primes

Test Set 1

Say $P(n)$ denotes the number of primes up to n, then the number of possible products of consecutive primes will be $P(n) - 1$. For this test set, we can find all the primes up to $n = 2021$ and generate the list of products of consecutive primes. Each query can then be answered by searching for the largest product which is still smaller than or equal to the given query.

Complexity analysis:

1. To generate primes, we can test all i up to n for primality by checking if any integer from 2 to $i - 1$ divides i. This takes $\sum_{i=0}^{n-2} i = O(n^2)$
2. Generating the list of products of consecutive primes from the prime list takes $O(n)$ (a tighter bound will be $O(P(n))$ but we do not require it for this case)
3. To answer all the queries, the complexity is $O(T \times n)$

So, the total complexity is $O(n^2) + O(T \times n)$ which is well under the required time limit for $n = 2021$.

Test Set 2

We do not need to generate all the primes up to n, but only until we find a prime which is greater than $\sqrt{n}$. The product of consecutive primes larger than $\sqrt{n}$ will be greater than n. For this test set, n can be up to 10^9, so we can generate primes up to 10^5 (rounding off $10^{4.5}$ to cover the one up case as well) using Sieve of Eratosthenes.

Complexity analysis: $O(\sqrt{n} \times \log(\log(\sqrt{n})))$ (for sieve) + $O(T \times P(\sqrt{n}))$ (for queries) which is well under the required time limit for $n = 10^9$.

Test Set 3

Our previous strategy of generating primes up to $\sqrt{n}$ will not work in this case as $\sqrt{n}$ can be as large as 10^9.

For any input n, there can be only two scenarios:

- The solution is the product of the largest prime smaller than or equal to $\sqrt{n}$ and the smallest prime greater than $\sqrt{n}$. Product of consecutive primes larger than that will be larger than n.
- The solution is the product of two largest primes smaller than or equal to $\sqrt{n}$. This will be less than n and any product of consecutive primes less than that will be even smaller.

So, we need primes only in the vicinity of $\sqrt{n}$, specifically in the worst case, 2 primes smaller than or equal to $\sqrt{n}$ and one prime larger than $\sqrt{n}$.

Now, how far out can the closest prime be from any given $\sqrt{n}$? This series of numbers is known as maximal prime gaps and for $\sqrt{n} = 10^9$ (for maximum possible $n = 10^{18}$), this number is 282 (called max_gap from now on). So, we can iterate backwards from $\sqrt{n}$ until we find 2 primes

and then iterate ahead from $\sqrt{n}$ for 1 more prime. In the worst case we would need to check integers in range $[\sqrt{n} - max\_gap \times 2, \sqrt{n} + max\_gap]$.

Complexity analysis: $O(\sqrt[4]{n} \times 3 \times max\_gap \times T)$
(primality check over $3 \times max\_gap$ possible integers for T test cases)

Note: The above complexity is not a tight bound. It's hard to hit the worst case scenario and even then most of the primality checks will terminate earlier (using just the first 7 primes, we can rule out 662 integers in any 3×282 range). A better primality test can be used to further drive the complexity down. Segmented sieve can also be used to have deterministic complexity - $O((\sqrt[4]{n} + 3 \times max\_gap \times T) \times \log(\log(\sqrt[4]{n})))$.

Kick Start 2021 - Round B

Analysis: Increasing Substring

Test set 1

A string X of length L is strictly increasing, if for every pair of indices i and j such that $1 \leq i < j \leq L$ (1-based), the character at position i is smaller than the character at position j. Using this, we can say that $X_i < X_{i+1}$ for $1 \leq i < L$. We can check this in $O(L)$ time complexity for string X.

There are $O(\mathbf{N}^2)$ substrings of string $\mathbf{S}$ each of which can have length at most $\mathbf{N}$. We can iterate over each substring and check whether it is strictly increasing in $O(\mathbf{N})$ time. If a substring from i to j is strictly increasing, update the length of longest strictly increasing substring ending at index j to $j - i + 1$ if it is greater than the previous found length. The overall time complexity of the solution is $O(\mathbf{N}^3)$.

Consider the following example with string $bbcd$. There is only 1 substring (b) ending at index 1. Hence, the answer for index 1 is 1. There are 2 substrings (b and bb) which end at index 2. bb is not strictly increasing string. Hence, the answer for index 2 is 1. There are 3 substrings (bbc, bc, and c) ending at index 3. bbc is not strictly increasing as b is repeated twice. bc is the longest strictly increasing substring ending at index 3. Hence, answer for index 3 is 2. There are 4 substrings ($bbcd$, bcd, cd, and d) ending at index 4. $bbcd$ is not strictly increasing as b is repeated twice. bcd is the longest strictly increasing substring ending at index 4. Hence, answer for index 4 is 3.

Sample Code(C++)

```
vector<int> longestStrictlyIncreasingSubstring(string S) {
  vector<int> answer(S.size(), 0);
  for(int i = 0; i < S.size(); i++) {
    for(int j = i; j < S.size(); j++) {
      bool is_strictly_increasing = true;
      for(int k = i; k < j; k++) {
        is_strictly_increasing &= (S[k] < S[k+1]);
      }
      if(is_strictly_increasing) {
        answer[j] = max(answer[j], j - i + 1);
      }
    }
  }
  return answer;
}
```

Test set 2

We cannot check each substring of string $\mathbf{S}$ for this test set due to the large constraints. We already know that, for a string $\mathbf{S}$ to be strictly increasing, $\mathbf{S_i} < \mathbf{S_{i+1}}$ for $1 \leq i < \mathbf{N}$. Consider that we have already calculated the length of the longest strictly increasing substring that ends at position i. Let this length be $MaxLen(i)$. Now we need to compute the answer for position $i + 1$. There is no need to consider all substrings ending at position $i + 1$. We can simply check if $\mathbf{S_i} < \mathbf{S_{i+1}}$. If this condition is satisfied, we can simply append S(i+1) to the longest increasing

substring ending at i and it would still be increasing and update $MaxLen(i+1) = MaxLen(i) + 1$. Otherwise, $MaxLen(i) = 1$. This way, we can calculate the length of the longest strictly increasing substring that ends at position i in constant time. Hence, the overall time complexity of the solution is $O(\mathbf{N})$.

Consider the following example with string $bbcda$. For index 1, $MaxLen(1) = 1$. For index 2, we can see that index 1 and index 2 have equal values and thus do not satisfy the constraint $\mathbf{S_i} < \mathbf{S_{i+1}}$. In this case, $MaxLen(2) = 1$. For index 3, $\mathbf{S_2} < \mathbf{S_3}$, hence we can extend the longest strictly increasing substring ending at index $\mathbf{2}$. In this case, $MaxLen(3) = 2$. For index 4, $\mathbf{S_3} < \mathbf{S_4}$, hence we can extend the longest strictly increasing substring ending at index $\mathbf{3}$. In this case, $MaxLen(4) = 3$. For index 4, $\mathbf{S_4} > \mathbf{S_5}$ which violates the condition for strictly increasing substring. In this case, $MaxLen(5) = 1$.

Sample Code(C++)

```
vector<int> longestStrictlyIncreasingSubstring(string S) {
  vector<int> answer(S.size(), 0);
  answer[0] = 1;
  for(int i = 1; i < S.size(); i++) {
      if(S[i - 1] < S[i]) {
        answer[i] = answer[i - 1] + 1;
      }
      else {
        answer[i] = 1;
      }
  }
  return answer;
}
```

Kick Start 2021 - Round B

Analysis: Longest Progression

Test Set 1

For any test case, if $\mathbf{N} = 2$, we can return 2 as the answer immediately, as any two numbers are trivially an arithmetic progression by themselves. For $\mathbf{N} \geq 3$, we can attempt a strategy that tries to improve it by considering every element of $\mathbf{A}$ as a possible starting point. For instance, let us try to start a longest progression beginning at $\mathbf{A_0}$. Let us call the first difference we encounter, $\mathbf{A_1} - \mathbf{A_0}$, d. d could be the common difference of a longest progression that starts from $\mathbf{A_0}$, or it could not be. We attempt both possibilities:

1. If it is, we just need to continue trying to extend our possible progression with $\mathbf{A_2}$, $\mathbf{A_3}$ and so on, checking that $\mathbf{A_2} - \mathbf{A_1} = d$, $\mathbf{A_3} - \mathbf{A_2} = d$, etc. Because we can change at most one number, the first time we find some k for which $\mathbf{A_k} - \mathbf{A_{k-1}} \neq d$, we fix $\mathbf{A_k}$ so that this expression does equal to d. We continue along the array afterwards until we reach the end or we find a second instance of k where $\mathbf{A_k} - \mathbf{A_{k-1}} \neq d$; at that point, we cannot go any further.
2. If it is not, the only possibility for a progression with length $>= 3$ starting from $\mathbf{A_0}$ is if the common difference is $(\mathbf{A_2} - \mathbf{A_0})/2$. In this case, we must set $\mathbf{A_1}$ such that $(\mathbf{A_1} - \mathbf{A_0}) = (\mathbf{A_2} - \mathbf{A_1})$. Hence, note that this is only possible if $\mathbf{A_2} - \mathbf{A_0}$ is an even number - if it is not, we can determine the longest progression starting at $\mathbf{A_0}$ under this scenario to simply be 2 (as we cannot extend it past the first two values of the array). Once we have done so, our common difference is set, and we continue trying to extend our progression starting from $\mathbf{A_3}$, $\mathbf{A_4}$, and so on, stopping at the first $\mathbf{A_k}$ where $(\mathbf{A_1} - \mathbf{A_0}) \neq (\mathbf{A_k} - \mathbf{A_{k-1}})$. At that point, the length of the longest progression starting at $\mathbf{A_0}$ under this scenario will be k.

Both possibilities 1 and 2 take $O(\mathbf{N})$ for a given starting point. Because we will try sequentially all possible starting points from $\mathbf{A_0}, \mathbf{A_1}, \ldots, \mathbf{A_{N-2}}$, finding the max value amongst all $\mathbf{N} - 1$ possible starting points with 2 possible scenarios each, our overall time complexity with this strategy will be $O(\mathbf{N}^2)$.

Test Set 2

Let's simplify the problem a little. The only thing that really matters in this situation is the common difference between two adjacent numbers, not the numbers themselves. That is, the length of the longest progression of $[a, b, c, d, e, f]$ is always going to be the same as that of $[a + k, b + k, c + k, d + k, e + k, f + k]$. So, we can create an array D where $D_i = \mathbf{A_{i+1}} - \mathbf{A_i}$. D_0 is hence $\mathbf{A_1} - \mathbf{A_0}$, and so on, until $D_{\mathbf{N}-2}$ which is $\mathbf{A_{N-1}} - \mathbf{A_{N-2}}$. For instance, given $\mathbf{A} = [1, 3, 4, 7, 9, 11]$, we find that $D = [2, 1, 3, 2, 2]$.

Now, we can break D into *chunks*. Each chunk is a contiguous portion of the array with identical values. For a given chunk, let us call this value its *common value*. For instance, for $D = [2, 1, 3, 2, 2]$, we can break this down as $[2], [1], [3], [2, 2]$, so 2 is the common value of the first chunk, and so on. Each chunk has a starting index within D. That index should be associated with the length and common value of that chunk, and this information stored for lookup. Then, we iterate through the chunks in ascending starting index order. For each chunk beginning at index i, with length k, and having common value d, we greedily try to fix the element in D right after the chunk, if it exists. Then, we also need a separate attempt to try fixing the element right before the chunk, if it exists. That means we will need to attempt merging for

each chunk up to two times. The following analysis is for what happens when we merge with the element *after* our current chunk, but the same applies for the corresponding situation when we merge backwards too.

The last element in our current chunk has index $(i + k - 1)$. If we greedily use our ability to change a number by *adding* some amount to D_{i+k} so it is equal to d, we must also reduce D_{i+k+1} by the same amount. The opposite case applies. At this point:

- if $D_{i+k} + D_{i+k+1} \neq 2 \times d$, then we have only been able to merge our current chunk with the element right after. The length of our extended chunk is now $k + 1$.
- if $D_{i+k} + D_{i+k+1} = 2 \times d$, but $D_{i+k+2} \neq d$, then changing D_{i+k} has allowed us to also merge with D_{i+k+1}. The length of our extended chunk is now $k + 2$.
- if $D_{i+k} + D_{i+k+1} = 2 \times d$, and $D_{i+k+2} = d$, then we have been able to merge even further with a chunk that starts 3 elements after our current one ends. The length of the whole merge becomes $k + 2+$ the length of the next chunk with common value d.

For instance, with $D = [1, 1], [2], [0, 0]$, if we try to merge forward from the first chunk which begins at index $i = 0$, has length $k = 2$ and common value $d = 1, D_2 + D_3 = 2 + 0 = 2 \times d$, but $D_4 \neq d$. This is the second scenario above, and the length of our extended chunk is now 4, so the overall progression length in $\mathbf{A}$ is 5. Another example: with $D = [2], [1], [3], [2, 2]$, if we try to merge forward from the first chunk which begins at index $i = 0$, has length $k = 1$ and common value $d = 2, D_1 + D_2 = 1 + 3 = 4 = 2 \times d$, and $D_3 = d$. This is the third scenario above, and the length of our extended chunk is now $1 + 2 + 2 = 5$, so the overall progression length in $\mathbf{A}$ is 6.

Our overall solution to the problem is then just the max of all the possible chunks made by considering each chunk in turn, and trying to merge it backwards and forwards. (As a result, if the test case in question falls into the third scenario listed above, the "longest progression" will actually be discovered twice, but this doesn't change its correctness.)

In summary, we run through the initial array once to create D, and then run through D to create map(s) of chunks tagged with the appropriate information, and then run through D again with our map of chunks to finally get our result. Assuming we have access to constant-time lookup for the lengths and common values of chunks, the overall time complexity is $O(\mathbf{N})$.

Kick Start 2021 - Round B

Analysis: Truck Delivery

Test Set 1

Let us root the tree at the capital city 1. For the small test set, we can answer each query by simply iterating the path from the given city $\mathbf{C_j}$ up to the capital city. We start out with the answer $ans = 0$, and whenever we encounter a road on the path with $\mathbf{L_i} \leq \mathbf{W_j}$, we update the answer to $ans = \gcd(ans, \mathbf{A_i})$.

The time complexity of the Greatest Common Divisor (GCD) operation $\gcd(a, b)$ is $O(\log(\min(a, b)))$ or $O(\log(MaxA))$ in our case, where $MaxA$ is the largest toll among all roads. A short proof of GCD time complexity is provided here, and it can be generalized to show that the amortized time complexity of a sequence of K GCD operations, where the result of a previous operation is fed into the next one, is $O(K + \log(MaxA))$ as opposed to $O(K \times \log(MaxA))$. Since a path in the tree can have up to $\mathbf{N}$ cities, the overall time complexity of the algorithm for all $\mathbf{Q}$ days is therefore $O(\mathbf{Q} \times (\mathbf{N} + \log(MaxA)))$.

Test Set 2

Since all queries are known in advance, we do not have to answer them in the given order, so let's group the queries by city $\mathbf{C_j}$.

Let's look at how we can answer all queries for a particular city C efficiently. First, we need to build a list of roads on the path from city C to the capital city 1 and sort them by the load-limits $\mathbf{L_i}$ in an increasing order. Let's also sort the queries for city C by weight $\mathbf{W_j}$ in a non-decreasing order. Now we can answer the queries by iterating these two lists in parallel and calculating GCD of all roads with load-limit up to and including the weight $\mathbf{W_j}$ of the current query.

The time complexity of this approach is $O(\mathbf{N}^2 \log(\mathbf{N}) + \mathbf{N} \log(MaxA) + \mathbf{Q} \log(\mathbf{Q}))$ as we need to sort the list of roads from each city to the capital city 1, perform a series of GCD operations for each of these $\mathbf{N}$ paths, and also have the queries sorted by loads.

Rather than building paths to the capital for each city independently, we can perform a Depth-first search (DFS) of the tree starting at the capital city 1 and answer all queries of a city C as we visit the city for the first time. That way, the cities and roads on the path from C to 1 are conveniently stored in the DFS stack. All we need is an efficient data structure that would store the toll $\mathbf{A_i}$ of precisely these roads and support GCD queries of tolls in the load-limit range $[1, \mathbf{W_j}]$.

That data structure happens to be a segment tree ST with load-limits $\mathbf{L_i}$ as keys (recall that all load-limits are unique), the tolls $\mathbf{A_i}$ as values, and GCD as the merge operation. Initially, the segment tree ST is empty, namely, the values of all its nodes are 0. Whenever we traverse the i-th road, we perform a point update operation $ST.update(\mathbf{L_i}, \mathbf{A_i})$, and, when we backtrack along this road in the DFS traveral, we cancel the value $\mathbf{A_i}$ by calling $ST.update(\mathbf{L_i}, 0)$. By doing so, we ensure that at the time of answering queries for a particular city, the segment tree ST contains the tolls of precisely the roads on the path to the capital city 1, and the answer to a query is $ST.query(1, \mathbf{W_j})$.

Let $MaxQ$ be the maximum load-limit among all roads. Each update or query of the segment tree involves $O(\log(MaxQ))$ GCD operations so the amortized time complexity of a single

update or query operation is $O(\log(MaxA) + \log(MaxQ))$. Since we have two update operations per road and one query operation per each day, the overall time complexity of the algorithm is $O((\mathbf{N} + \mathbf{Q}) \times (\log(MaxA) + \log(MaxQ)))$.